

Introduction to ensemble algorithms

Bagging vs. Boosting: key concepts and differences

Lucas DEREGNAUCOURT

Specially-appointed researcher
D3 Center, The University of Osaka, Japan.
lucas@is.ids.osaka-u.ac.jp

- I. Foundations of Classification & Ensemble Learning
 - A. Notations & Decision trees
 - B. Ensemble Learning: How weak learners create a strong model
 - C. Aggregation strategies for decision-making
- II. Bagging strategy
 - A. Bootstrap aggregation: key concepts
 - B. Beating the trees with Random Forests
 - C. Comparison with Extra Trees
- III. Boosting strategy
 - A. Sequential learning & bias reduction
 - B. Adaptive weighting with AdaBoost
 - C. Modern Gradient Boosted trees: XGBoost & LightGBM
- IV. Summary and conclusion
 - A. Comparative matrix: Bagging vs. Boosting
 - B. How to choose the right model ?

Table of contents

Foundations of Classification & Ensemble Learning

Classification: objective and notations

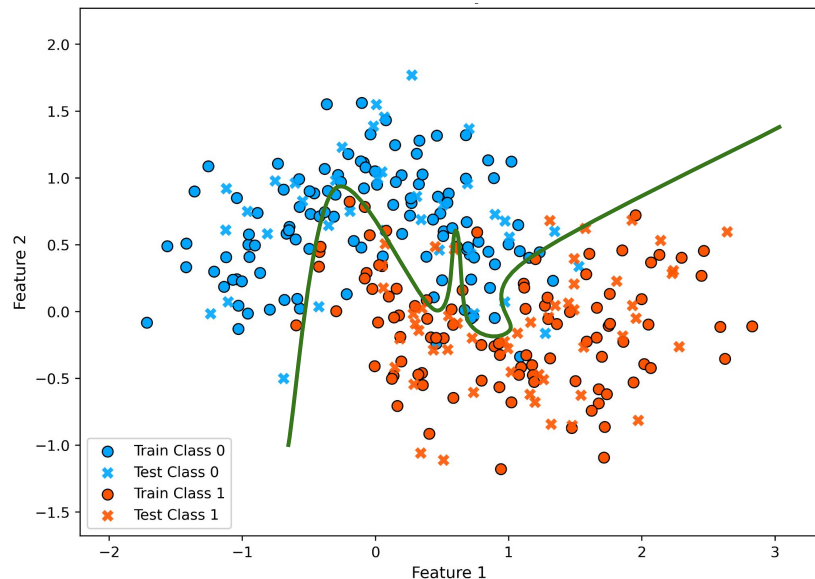
Let D be a dataset composed of N annotated data $\{d_i\}_{i=1,\dots,N} = \{x_i, y_i\}_{i=1,\dots,N}$.

Each sample x_i has M features ($x_i = \{F_j\}_{j=1,\dots,M}$) and is associated to a label y_i among the K possible classes ($y_i \in \{C_k\}_{k=1,\dots,K}$).

Objective: Given a new sample x_{N+1} , we want to predict its label $y_{N+1} \in \{C_k\}_{k=1,\dots,K}$.

How: Find some rules to separate the dataset, then evaluate the generalization on test data.

Classification: a simple example



How should we define the separation rules?

Moons dataset (sklearn)

First solution:

Define a function:

- 2 classes. 2 features
- 150 points for each class

$$y_i = \text{sign}(f(F_{i_1}, F_{i_2}))$$

Example: logistic regression

Question: How should we define the separation rules?

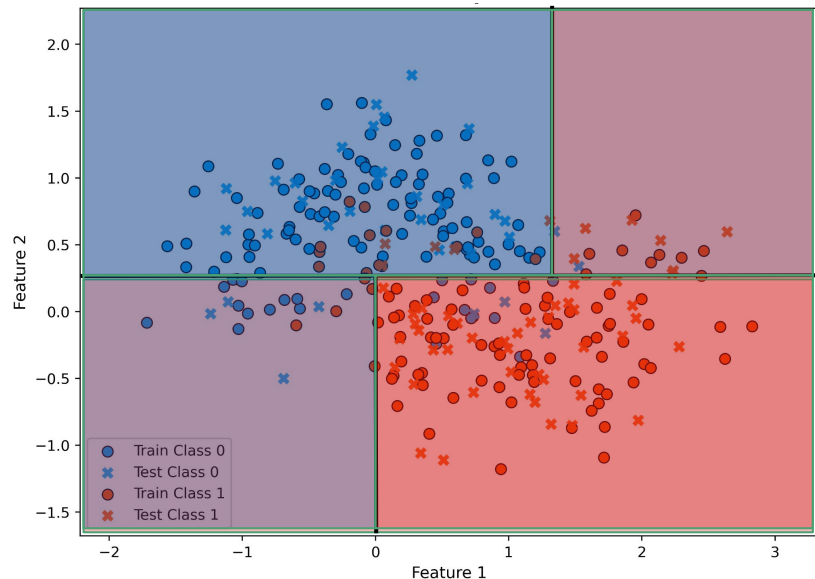
$$p(y_i = 1|x_i) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 F_1 + \beta_2 F_2)}}$$

How to find β_0, β_1 and β_2 ? We optimize by minimizing the log-likelihood.

If $p(y_i = 1|x_i) > 0.5$, then we predict the Class 1.
Else, we predict the Class 0.

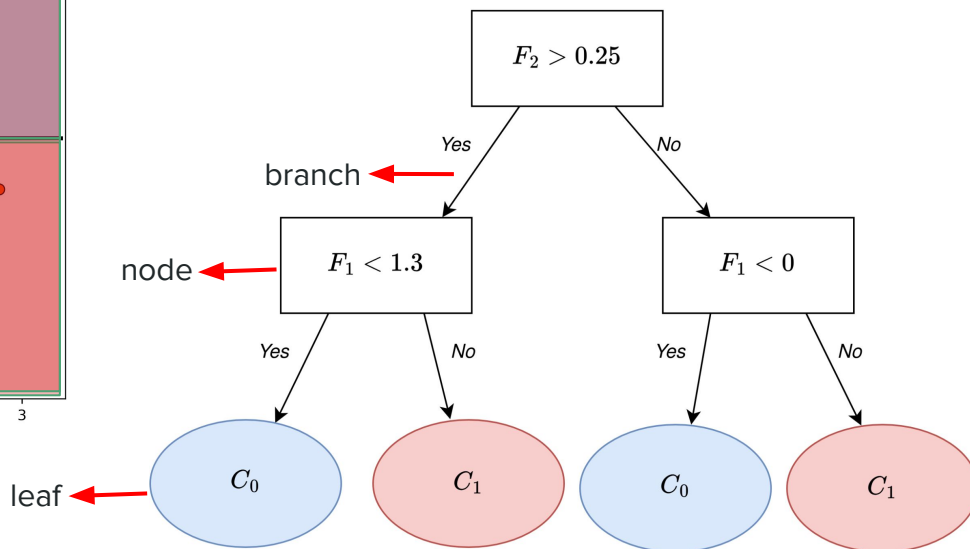
Classification: a simple example

How should we define the separation rules?



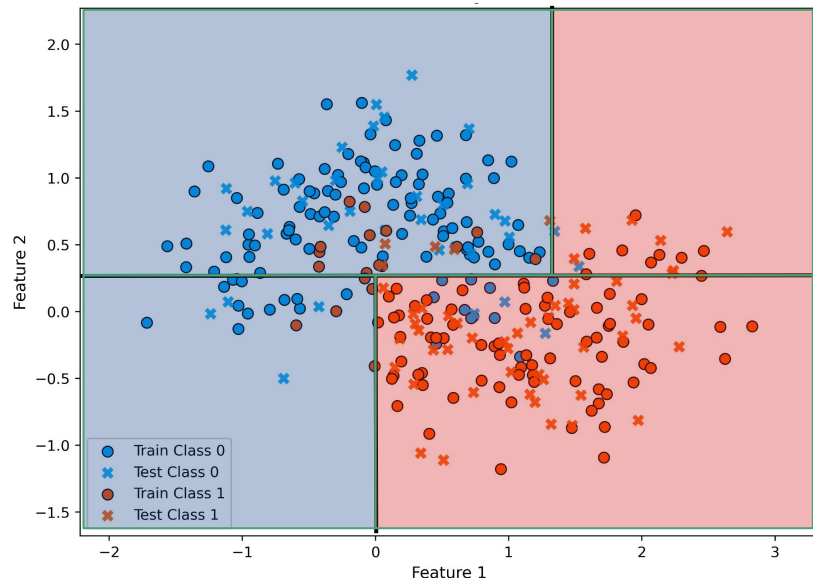
Second solution:

Define a set of sequential rules:



Classification: a simple example

How to evaluate the model ?



Accuracy:
$$Acc = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\hat{y}_i = y_i}$$

During training, the model optimizes the accuracy on the train set.

To evaluate the generalizability of the trained model, we compute the accuracy on the test set.

If $Acc_{train} \gg Acc_{test}$, then the model **overfitted** the training data.

With the previous tree:

$$Acc_{train} = 87.62\%$$

$$Acc_{test} = 90\%$$

→ Even a small tree can achieve good results!

Classification and regression trees (CART) algorithm

Breiman *et al.*, 1984.

Algorithm: CART Training Algorithm (Classification & Regression)

Input : Dataset $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, max depth d_{max} , min samples n_{min}

Output : Decision Tree Root Node T

function BuildTree(D , current_depth) :

```
    if current_depth  $\geq d_{max}$  or  $|D| < n_{min}$  or Impurity( $D$ ) = 0 then
        Create leaf node  $L$ 
        Prediction( $L$ )  $\leftarrow$  MajorityClass( $D$ ) // or Mean( $y$ ) for regression
        return  $L$ 
    end if
```

“Stop” condition

```
    best_split  $\leftarrow$  null, max_gain  $\leftarrow -\infty$ 
```

```
    for each feature  $j \in \{1, \dots, p\}$  do
```

```
        for each unique threshold  $k$  in feature  $j$  do
```

```
             $D_L \leftarrow \{(\mathbf{x}, y) \in D \mid x_j \leq k\}$ ,  $D_R \leftarrow \{(\mathbf{x}, y) \in D \mid x_j > k\}$ 
```

```
             $\Delta I \leftarrow$  Impurity( $D$ ) -  $\left(\frac{|D_L|}{|D|} \text{Impurity}(D_L) + \frac{|D_R|}{|D|} \text{Impurity}(D_R)\right)$ 
```

```
            if  $\Delta I > \text{max\_gain}$  then
```

```
                max_gain  $\leftarrow \Delta I$ , best_split  $\leftarrow (j, k, D_L, D_R)$ 
```

```
            end if
```

```
        end for
```

```
    end for
```

Possible splits comparison

```
    Create internal node  $N$  with split condition  $x_j \leq k$ 
```

```
    LeftChild( $N$ )  $\leftarrow$  BuildTree( $D_L$ , current_depth + 1)
```

```
    RightChild( $N$ )  $\leftarrow$  BuildTree( $D_R$ , current_depth + 1)
```

```
    return  $N$ 
```

Create the node that maximized the Gini impurity.
Then recursively apply the algorithm to the left and right branches of the node.

Gini impurity

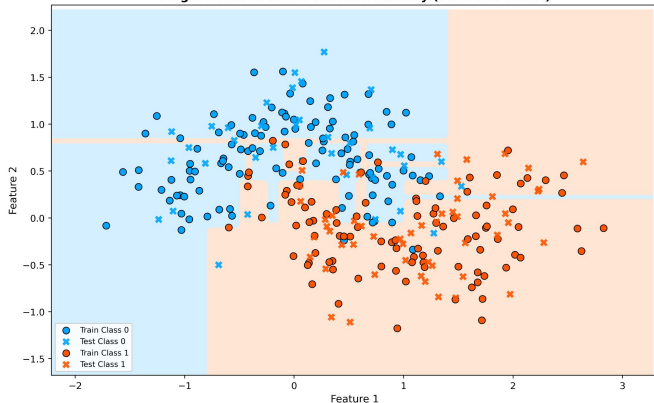
$$\text{Impurity}(D) = 1 - \sum_{k=1}^K p_k^2$$

$$p_k = \frac{|D_k|}{|D|} = \frac{\sum_{i \in D} \mathbf{1}_{(y_i=k)}}{|D|}$$

CART algorithm on the simple example

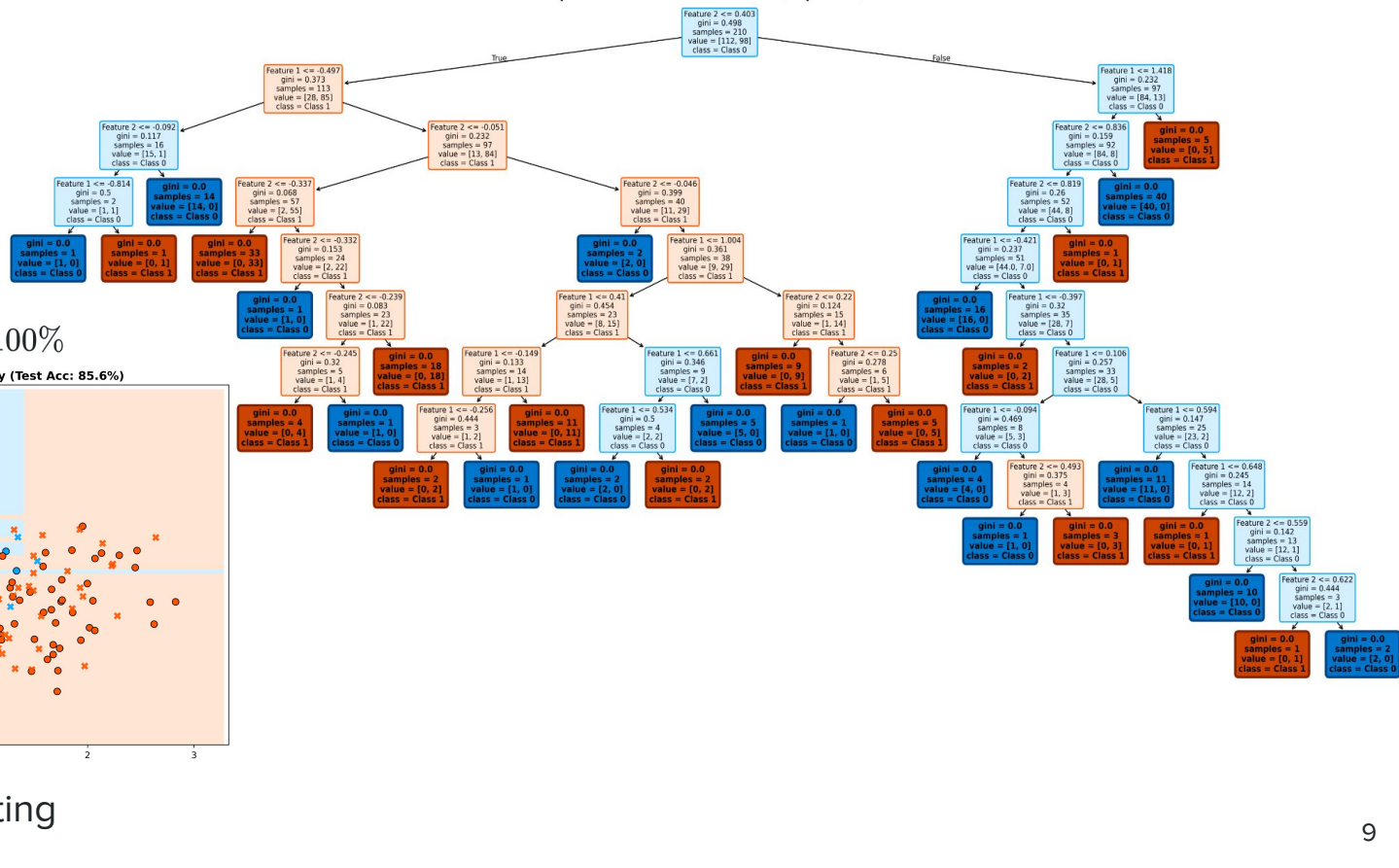
$Acc_{train} = 100\%$

Single Decision Tree Decision Boundary (Test Acc: 85.6%)



→ clear overfitting

Optimal Decision Tree Structure (Depth: 11)



Ensemble Learning: weaker is stronger ?

Weak learner: A model that performs only slightly better than random guessing (e.g., a decision tree with a low depth).

Main weakness of the “strong” classification tree: overfitting on the training data, poor generalization capacity on the test data.

Our manual tree achieved a better test accuracy than the “optimal” one:

$Acc_{test} = 90\%$ VS $Acc_{test} = 85.6\%$, but it was weaker on the training data.

Mathematical Foundation (Condorcet's Jury Theorem): If each independent model has an accuracy over 50%, the probability of a correct majority vote approaches 1.0 as the number of models $N \rightarrow \infty$.

Creating and aggregating weak learners

The models should be **independent**: the CART algorithm is deterministic, so training multiple trees on the same data with the same parameters (e.g. max depth) will result in the same tree. Then, the aggregation is useless.

How do we guarantee independence ?

- **Features selection**: each tree is trained on a different sets of features.
- **Data selection**: each tree is trained on a different training subset.

How do we make a decision with multiple models ?

1. Each model makes its decision (1 predicted class per model).
2. The majority class wins !

Question: what if there is no majority?

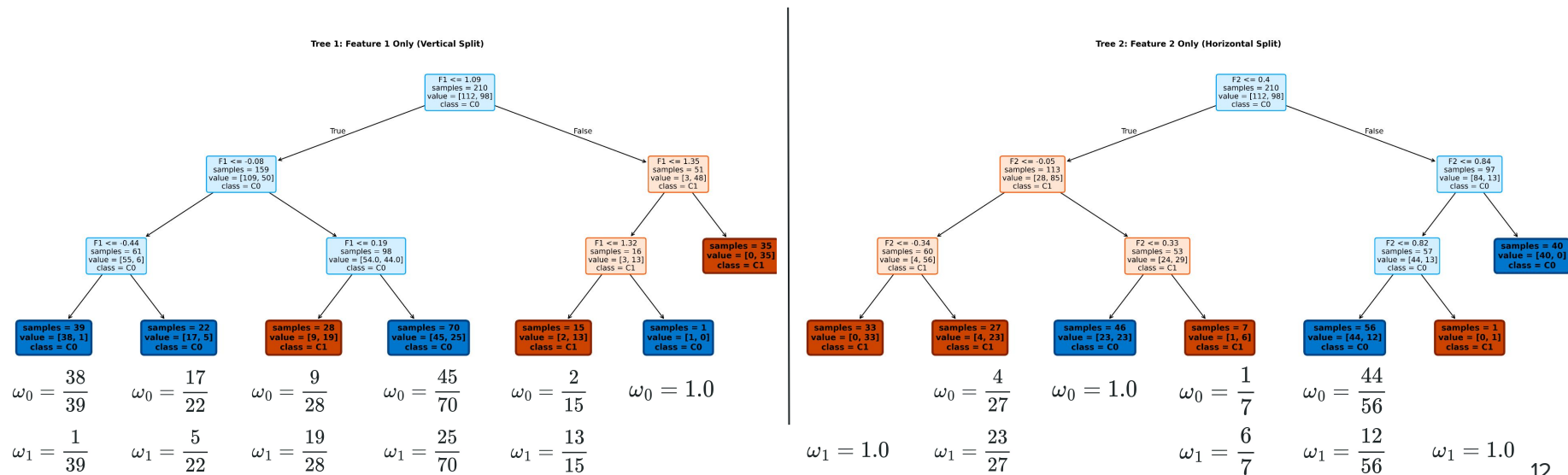
Ensuring a majority with Soft Voting

Idea: The last leaves for each model often have a non-zero Gini impurity value.

→ The class proportions can be regarded as probabilities.

→ We can apply a weighted sum for the aggregated decision-making!

$$\hat{y} = \arg \max_{c \in \{1, \dots, K\}} \left(\frac{\sum_{m=1}^M \omega_m \cdot P_m(y = c \mid \mathbf{x})}{\sum_{m=1}^M \omega_m} \right)$$



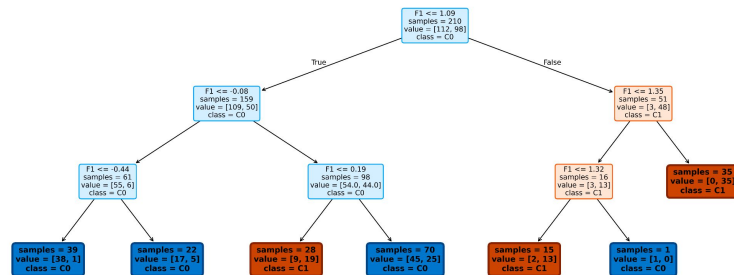
Ensemble learning on the simple example

3 independent models:

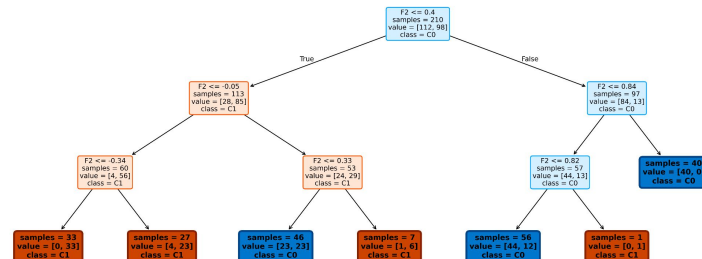
- CART 1: trained with only the Feature 1.
- CART 2: trained with only the Feature 2.
- CART 3: trained with 20% of the data.

max depth = 3

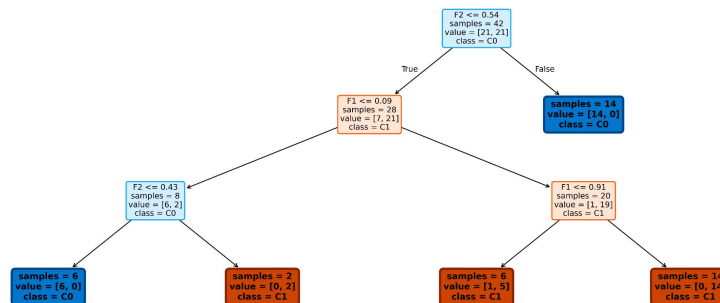
Tree 1: Feature 1 Only (Vertical Split)



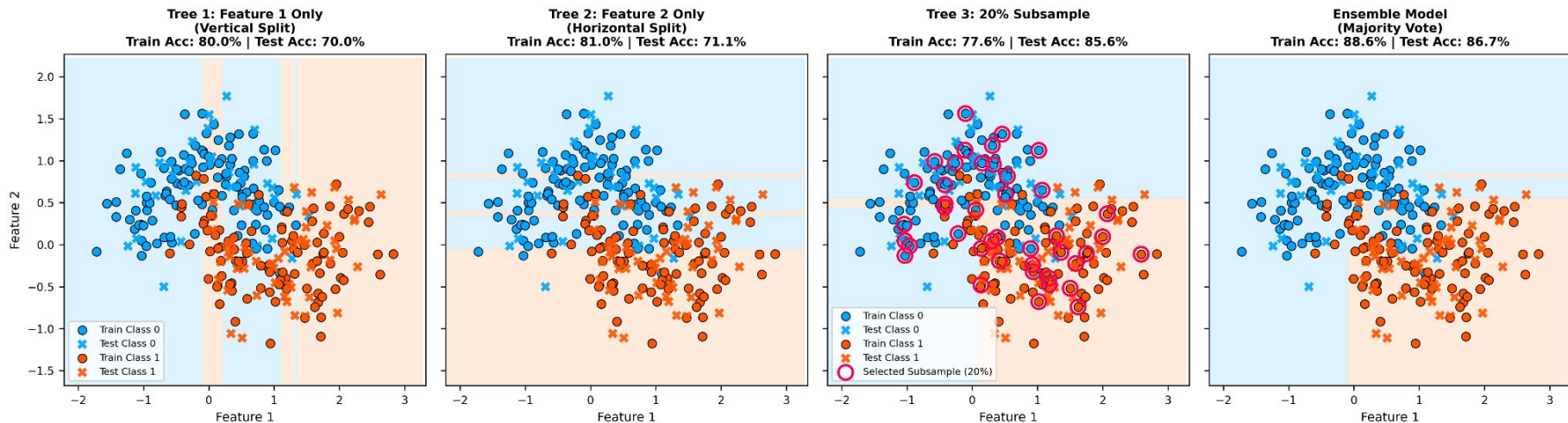
Tree 2: Feature 2 Only (Horizontal Split)



Tree 3: 20% Subsample



Ensemble learning on the simple example



The “manual” setup to guarantee the independence of the models is not very efficient.

- Maybe more trees are needed? Or smarter ways to build weak learners?
- Even with only 20% of the data, the random subsample achieves the best results.

➔ Randomness may be the key to success...

Bagging strategy

The art of bootstrapping

Reminder:

- We want a principled method to build independent models.
- The random sampling achieved better results than the features splits.

What if we build weak learners with only random sampling?

The art of bootstrapping

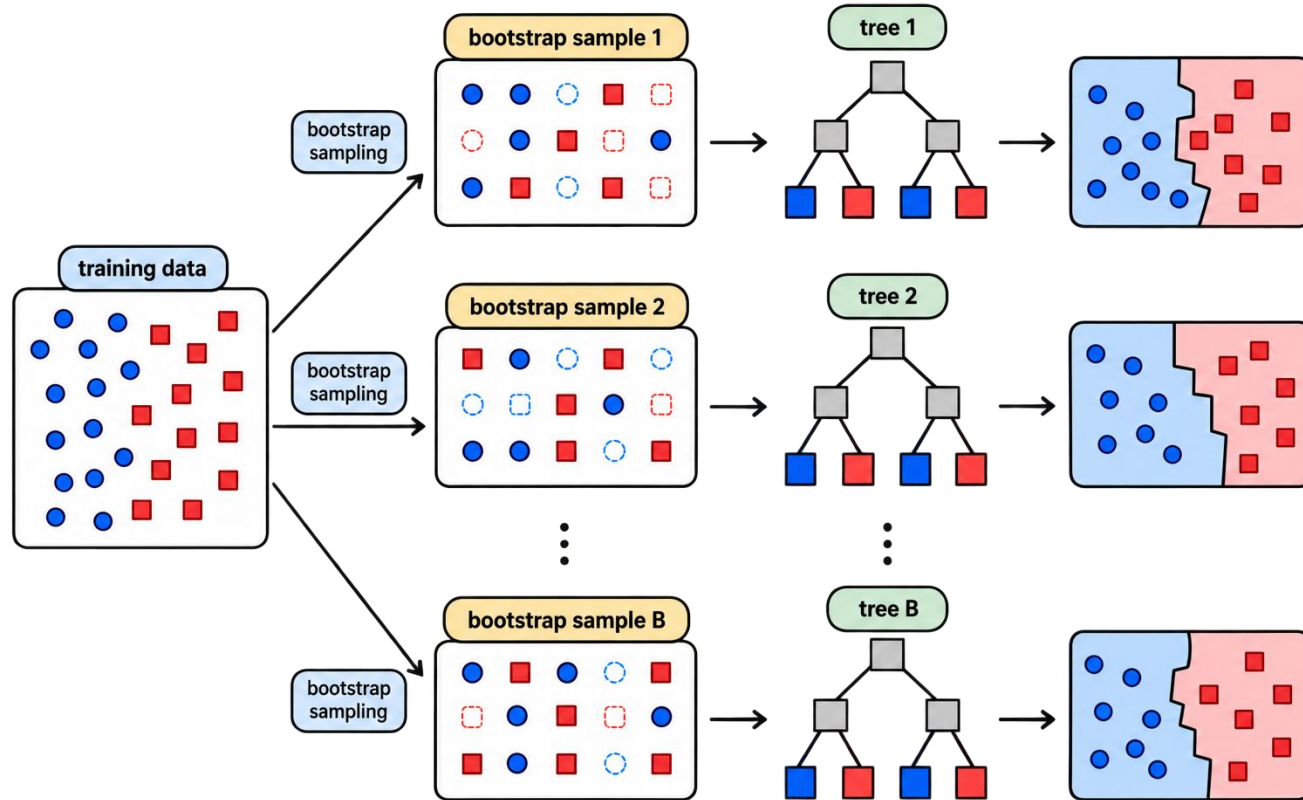
Motivation: A single decision tree can be **highly sensitive** to **ambiguous samples**.

Objective: Create a set of **independent** “small” trees (*weak learners*) by **randomly sampling** (with replacement) a subset of the training data.

Theoretical advantage: Aggregating diverse trees **reduces the variance** of the final model and **improves its stability**.

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B f_b(x) \right) = \frac{\sigma^2}{B}$$

The art of bootstrapping



Bootstrap + Aggregation = **Bagging**

Breiman, 1984.

Algorithm: Bagging (Bootstrap Aggregation)

Input : Dataset $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, number of trees B

Output : Ensemble classifier $\{T_1, \dots, T_B\}$

for each $b \in \{1, \dots, B\}$ **do**

$D_b \leftarrow$ Draw n observations from D with replacement

$T_b \leftarrow \text{BuildTree}(D_b, 0)$

} Bootstrap step

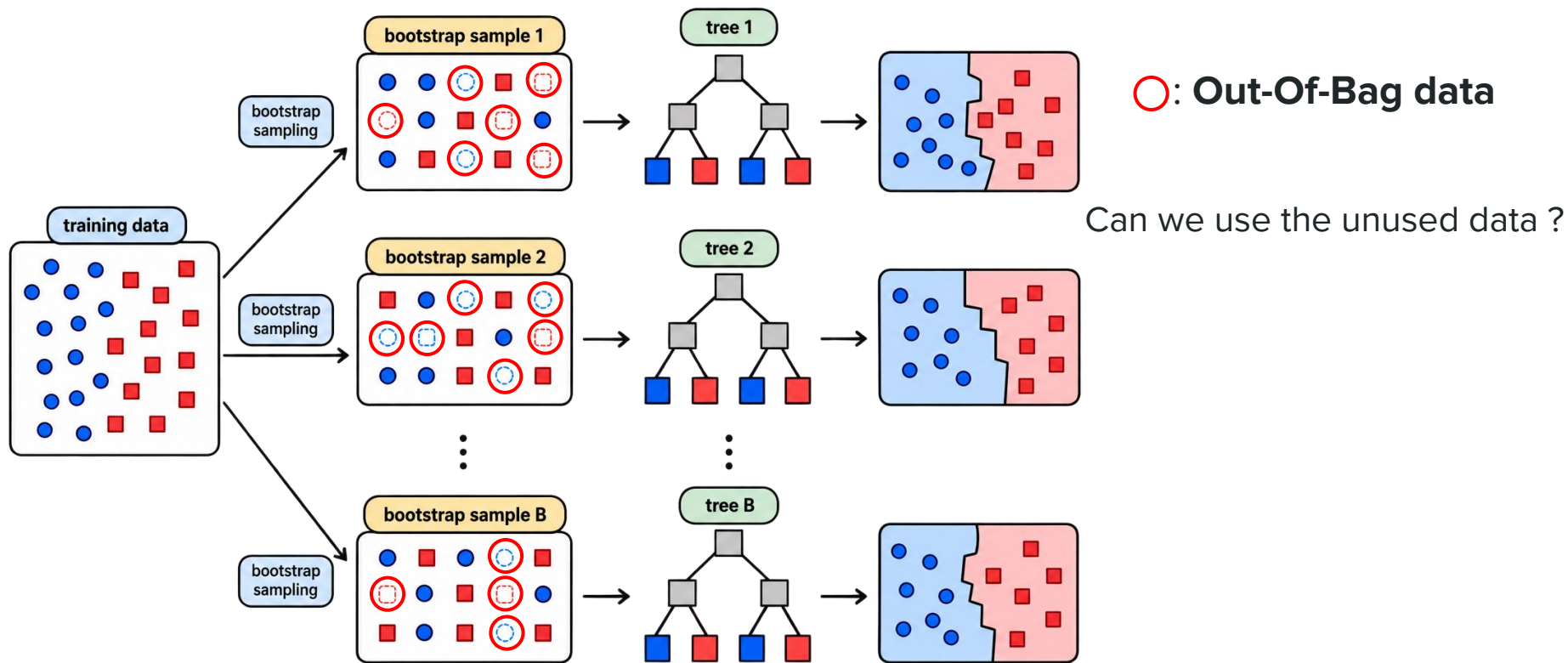
end for

Prediction :

$\hat{y}(\mathbf{x}) \leftarrow \text{MajorityVote}(T_1(\mathbf{x}), \dots, T_B(\mathbf{x}))$

} Aggregation step

Out-Of-Bag (OOB) data



Evaluating the generalizability with OOB data

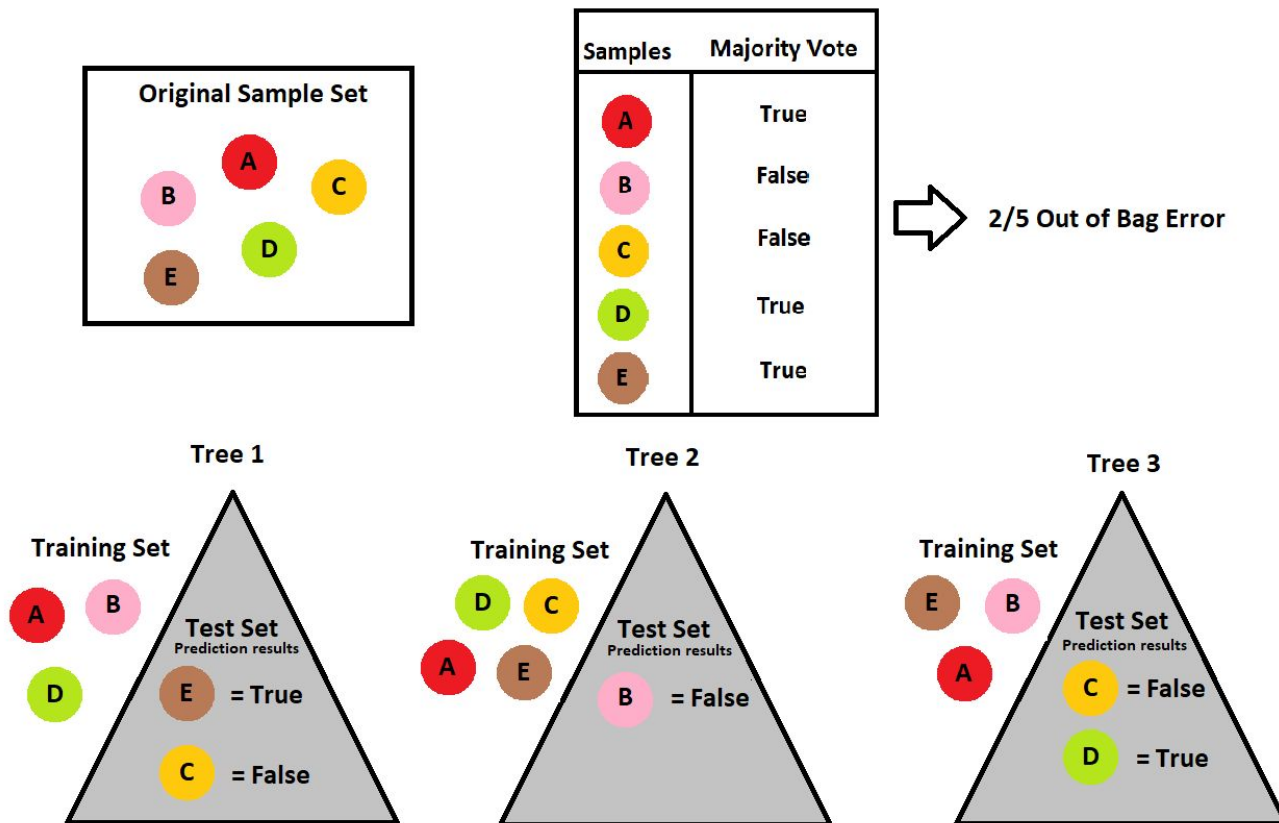
How does it work?

- For each sample x_i , aggregate predictions *only* from the trees trained without x_i in their bootstrap samples.
- Compare these aggregated out-of-bag predictions against the true labels across all dataset samples to calculate the OOB error.

Advantages

- With bootstrap strategy, the entire dataset (including the test!) can be used.
- Evaluating the accuracy on the OOB data provides an unbiased estimator of how well the model generalizes.

Evaluating the generalizability with OOB data



Improving bagging with Random Forest

Forest: An ensemble of decision tree models whose individual predictions are combined to produce a single, more accurate output.

Motivation: Standard Bagging produces highly correlated trees when dominant features exist.

How: First, a bootstrap step is applied. Then, at each split in every tree, the algorithm restricts candidate splits to a randomly chosen subset of features.

Random Forest

Breiman, 2001.

Algorithm: Random Forest

Input : Dataset $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, number of trees B ,
number of features per split $m < p$

Output : Ensemble classifier $\{T_1, \dots, T_B\}$

for each $b \in \{1, \dots, B\}$ **do**

$D_b \leftarrow$ Draw n observations from D with replacement

 Grow tree T_b from D_b by recursively repeating for each node:

1. $\mathcal{F} \leftarrow$ Select m features at random from $\{1, \dots, p\}$
2. Pick best feature $j \in \mathcal{F}$ and threshold k to split node
3. Split node into two child nodes

end for

Prediction :

$\hat{y}(\mathbf{x}) \leftarrow \text{MajorityVote}(T_1(\mathbf{x}), \dots, T_B(\mathbf{x}))$

} Bootstrap step

} Random features selection

} Minimize the Gini impurity

} Aggregation step

Extra Trees: extremely randomized trees

Geurts et al., 2006.

Motivation: Random Forests reduce variance by sampling features at each node, but searching for the optimal split threshold among all possible values is still computationally expensive and can overfit noisy training data.

How it works:

- **No bootstrapping:** Each tree in the ensemble is trained on the entire original dataset rather than a bootstrap sample.
- **Random threshold selection:** At each node, m features are selected at random. For **each** of these m features, a single split threshold is drawn **uniformly at random** between the minimum and maximum values of that feature in the current node.
- **Selection:** The algorithm evaluates the Impurity Gain across these m random candidates and picks the best one to split the node.

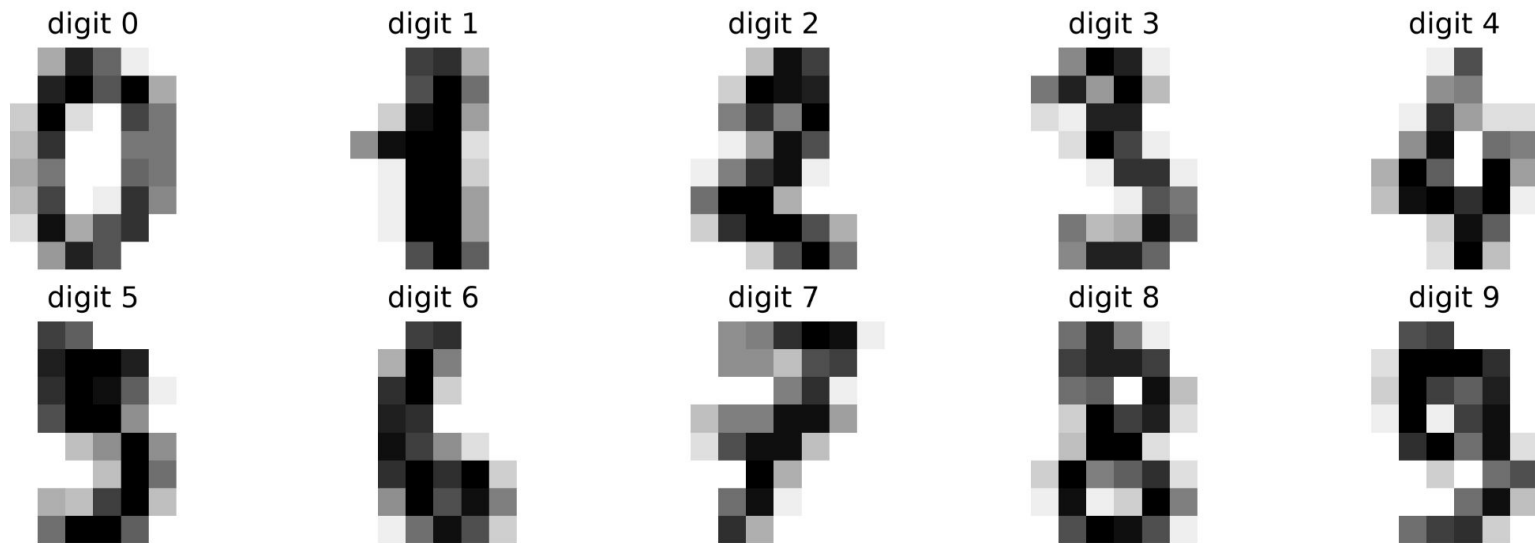
Some practical advices

Dataset Scenario	Dimensions	Recommended Model	Hyperparameters
Standard	$N \in [10^2, 10^4], p \in [5, 50]$	Random Forest	$B = 100\text{--}200, m = \sqrt{p}, \text{min_leaf} = 1$
High Feature Count ($p > N$)	$N < 1,000, p \in [100, 1,000]$	Random Forest	$B = 250\text{--}500, m = \sqrt{p}, \text{min_leaf} = 1\text{--}5$
Large Scale ($N \gg p$)	$N > 100,000, p \in [20, 200]$	ExtraTrees	$B = 100\text{--}300, m = \sqrt{p}, \text{min_leaf} = 10\text{--}50$
Noisy Data	Any N , high variance	Both	$B = 300\text{--}500, m = \sqrt{p}, \text{min_leaf} = 5\text{--}20$

min_leaf: The minimum number of training samples required to form a terminal node (leaf) in a decision tree. During tree construction, a proposed split is rejected if either resulting child node would end up with fewer samples than this threshold.

Example: Digit dataset

Handwritten digits - 8x8 grayscale (pixel values 0-16)



1797 samples (1258 for training and 539 for testing).

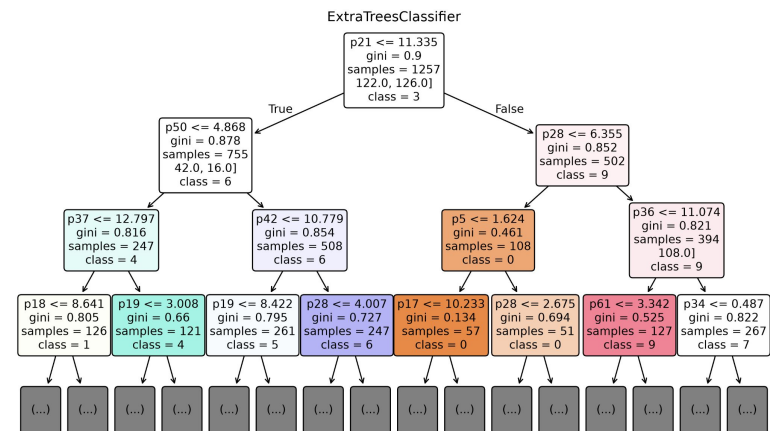
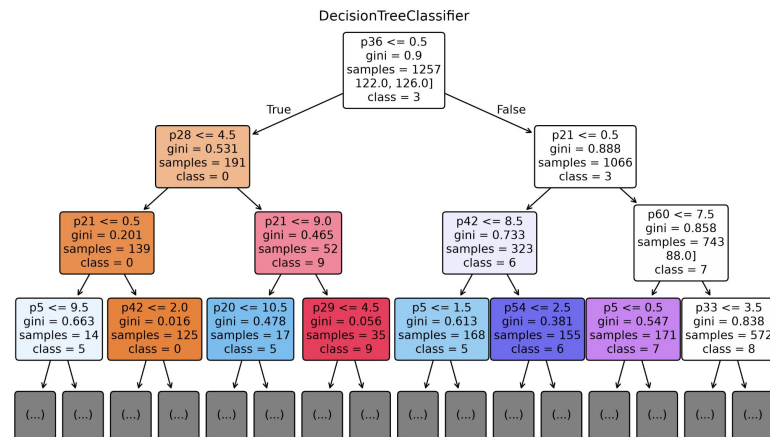
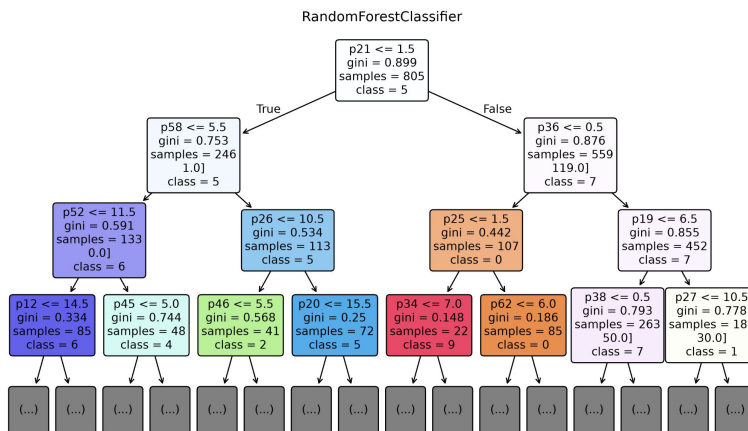
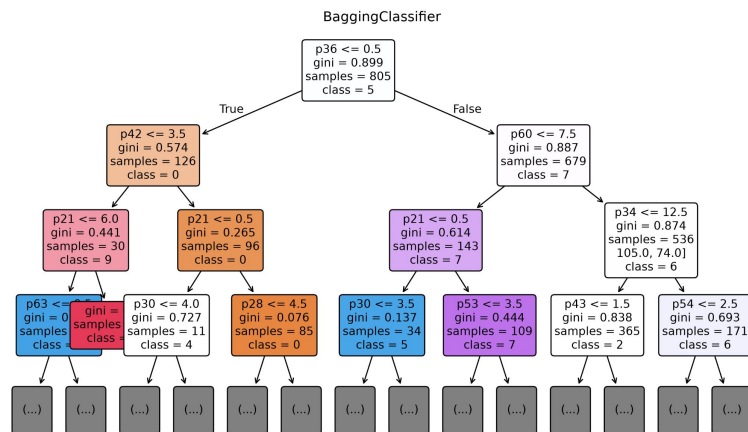
$B = 100$, $min_leaf=1$, $m=8$

64 features (1 per pixel).

Example: Digit dataset

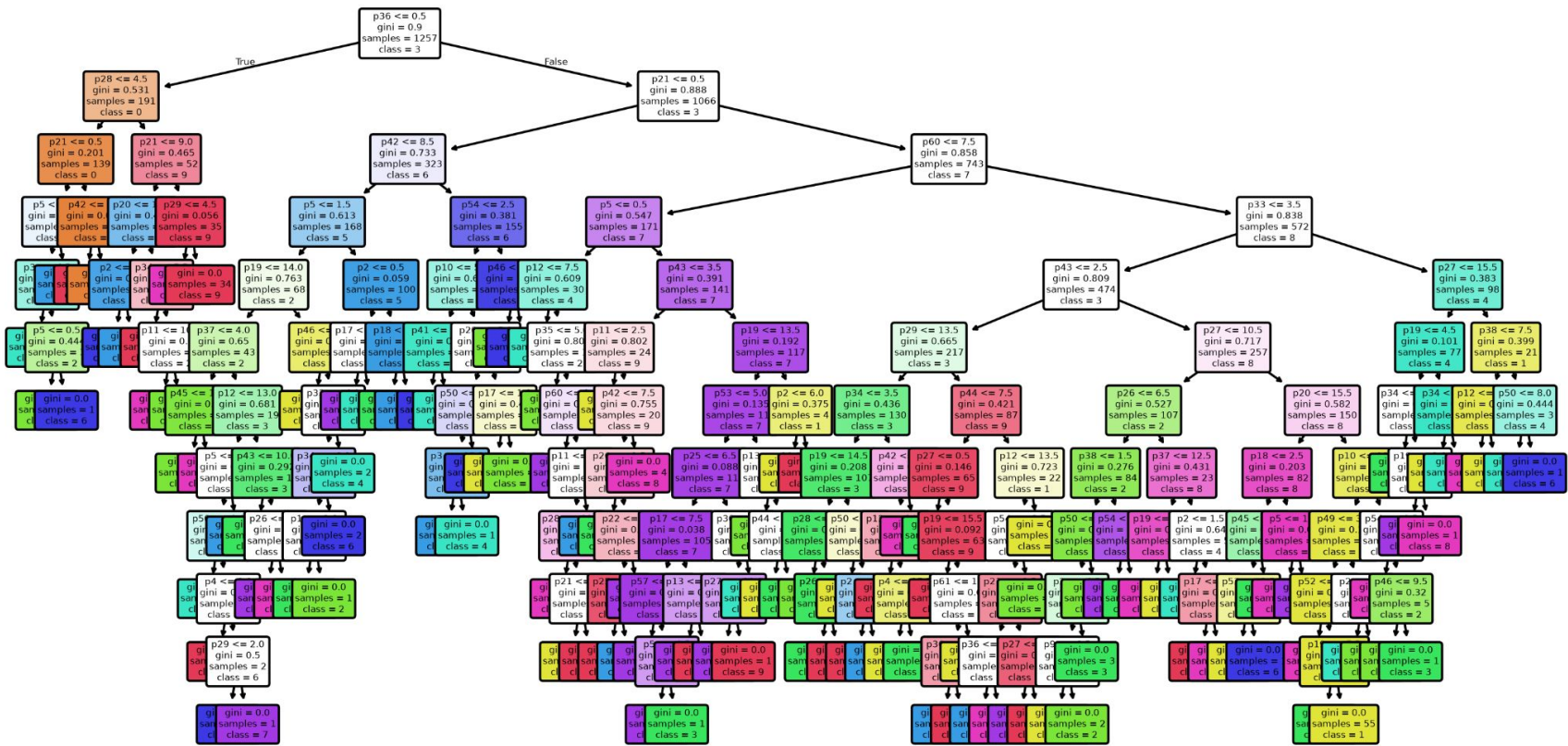
	CART	Bagging	Random Forest	Extra Trees
train acc.	100%	100%	100%	100%
test acc.	85.2%	92,2%	96.7%	97.4%
OOB error	-	4.38%	2.70%	-

Example: Digit dataset

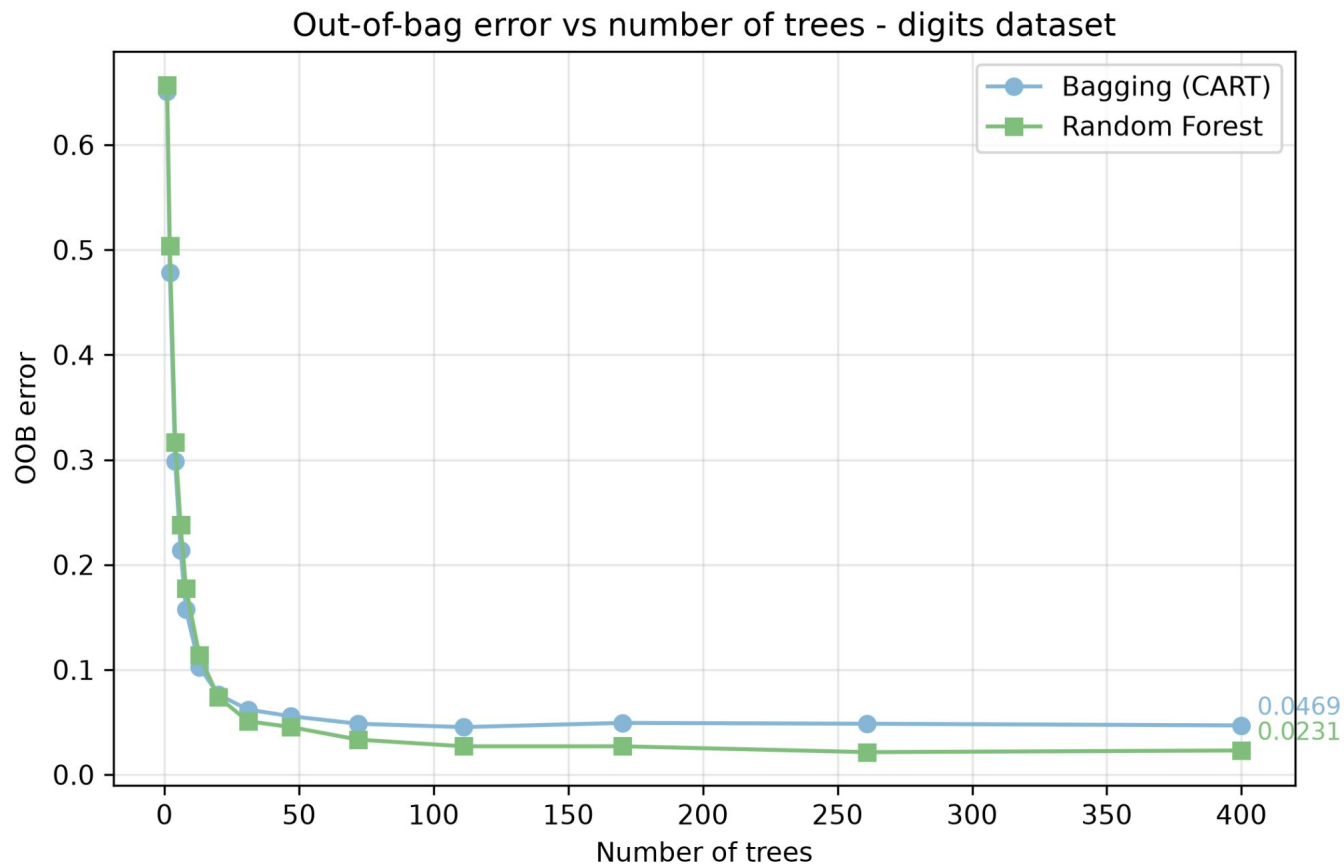


Example: Digit dataset

Maximal CART tree (full: 11 levels, 121 leaves) - 1257 training samples



Example: Digit dataset



Boosting strategy

Shifting from bagging to boosting

Reminder: With bagging strategy, we built many trees independently in parallel. We relied on the "*wisdom of the crowd*" (**Condorcet's Jury**) through simple averaging to outperform individual model variance without touching bias.

The Problem: Averaging independent models cannot fix systematic underfitting or uncaptured structural patterns.

Boosting (Sequential): Trains low-capacity weak learners iteratively. Every new tree targets the residual errors of the existing ensemble.

AdaBoost

Freund and Schapire, 1994.

Theoretical Origin (The Weak Learning Hypothesis): In PAC (Probably Approximately Correct) learning theory, Schapire (1990) proved that a collection of *weak learners* can be combined to construct a single "*strong learner*" with higher accuracy.

Overview: AdaBoost converts a sequence of weak, high-bias estimators (trees with a single node) into a single highly accurate model by systematically forcing each new learner to fix the errors of its predecessor.

How it works

- **Dynamic Instance Re-Weighting:** Samples misclassified in the current iteration gain higher importance weights, forcing the next weak learner to prioritize hard-to-predict data points.
- **Performance-Based Voting:** Each weak learner receives an influence score proportional to its individual accuracy, ensuring stronger models carry more weight in predictions.
- **Weighted combination:** The final classification is decided by taking a weighted majority vote.

AdaBoost algorithm

	Algorithm: AdaBoost
	Input: Dataset $\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N)\}$, $y_i \in \{-1, +1\}$ Weak learning algorithm $\mathcal{L}$ Number of iterations M
	Output: Ensemble classifier $H(x)$
weights uniform initialization	{ Initialize observation weights: $w_i^{(1)} = \frac{1}{N}$, $\forall i \in \{1, \dots, N\}$
Train the first CART and compute its error	{ For $m = 1$ to M do : Fit weak classifier $h_m(x) \leftarrow \mathcal{L}(\mathcal{D}, w^{(m)})$ where $h_m(x_i) \in \{-1, +1\}$ Compute weighted misclassification error: $\epsilon_m = \sum_{i=1}^N w_i^{(m)} \cdot \mathbb{I}(y_i \neq h_m(x_i))$ If $\epsilon_m \geq 0.5$ then break Compute estimator voting weight: $\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right)$ Update and renormalize sample weights for $i = 1, \dots, N$: $w_i^{(m+1)} = \frac{w_i^{(m)} \cdot \exp(-\alpha_m y_i h_m(x_i))}{Z_m}$ where $Z_m = \sum_{j=1}^N w_j^{(m)} \cdot \exp(-\alpha_m y_j h_m(x_j))$
Compute the weak learner's voting weight and update the data weights	{
	End For
	Return $H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(x) \right)$

AdaBoost

Uniform Weight Initialization

Set equal starting importance across all N training samples:

$$w_i^{(1)} = \frac{1}{N}, \quad \text{for } i = 1, \dots, N$$

AdaBoost

Evaluating Weighted Learner Error

Train weak estimator $h_m(x)$ on weighted data.

Compute the importance-weighted misclassification rate:

$$\epsilon_m = \sum_{i=1}^N w_i^{(m)} \cdot \mathbb{I}(y_i \neq h_m(x_i))$$

ϵ_m represents the sum of sample weights for all incorrectly classified instances.

AdaBoost

Estimator Voting Weight

Calculate model weight α_m depending on its error ϵ_m :

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right)$$

Behavior: As $\epsilon_m \rightarrow 0$, $\alpha_m \rightarrow +\infty$

AdaBoost

Data weights update

Up-weight misclassified points and down-weight correct ones:

$$w_i^{(m+1)} = \frac{w_i^{(m)} \exp(-\alpha_m y_i h_m(x_i))}{Z_m}$$

Renormalization factor Z_m keeps weights summing to 1:

$$Z_m = \sum_{i=1}^N w_i^{(m)} \exp(-\alpha_m y_i h_m(x_i))$$

AdaBoost

Final ensemble aggregation

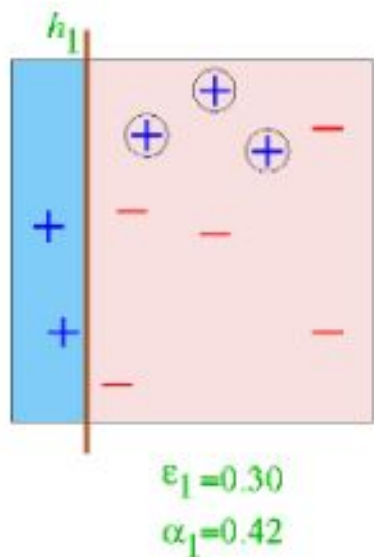
Combine all stage-wise weak learners into a single output:

$$H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(x) \right)$$

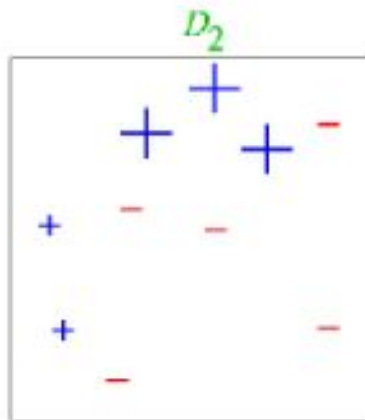
If $K > 2$, the aggregation is done via a *Soft Voting* with the α_m weights. However, AdaBoost is not well suited for classification with > 2 classes (see the SAMME algorithm for a generalized AdaBoost).

AdaBoost

A small example



2 classes, 5 samples for each.

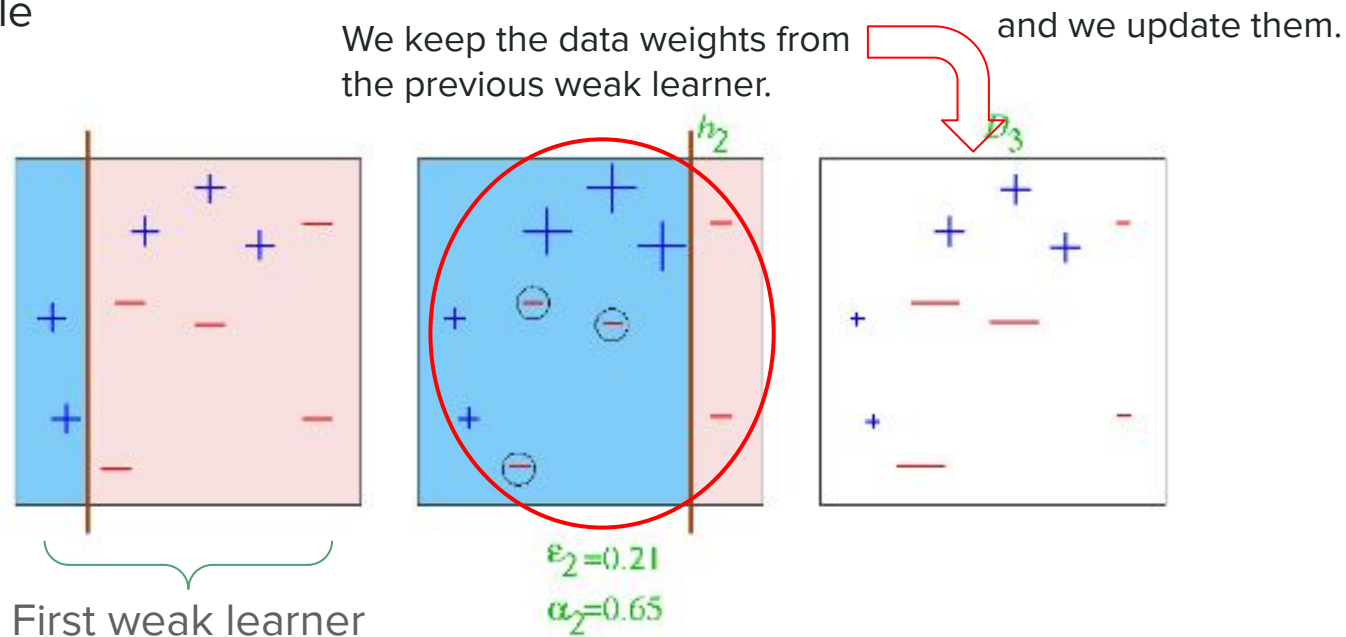


3 misclassified
→ error rate = 30%

The weights are increased for the misclassified data and decreased for the correctly classified ones.

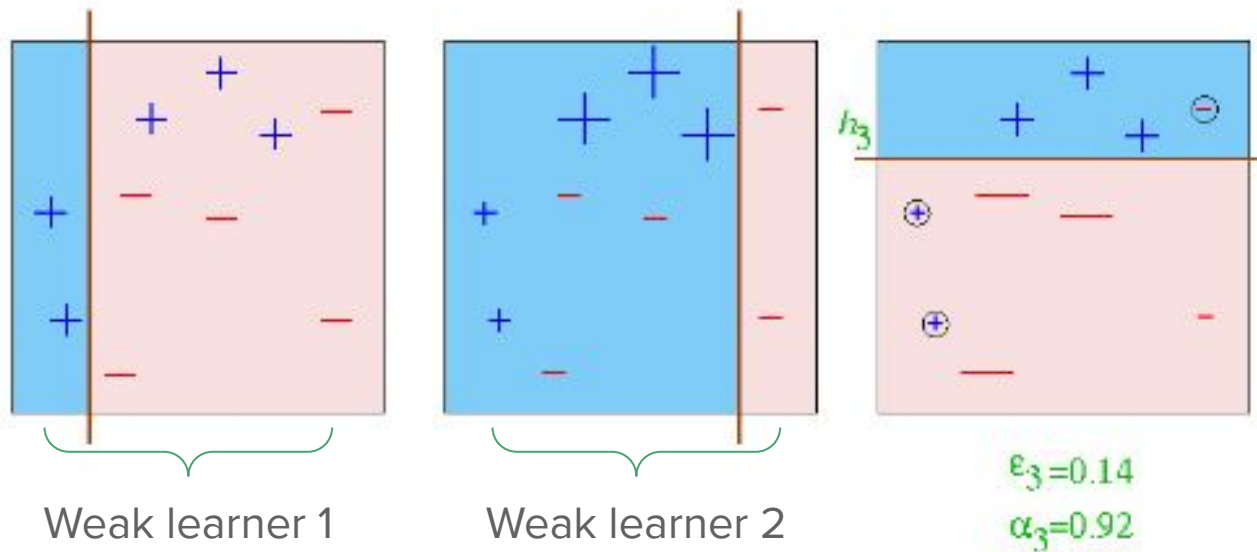
AdaBoost

A small example



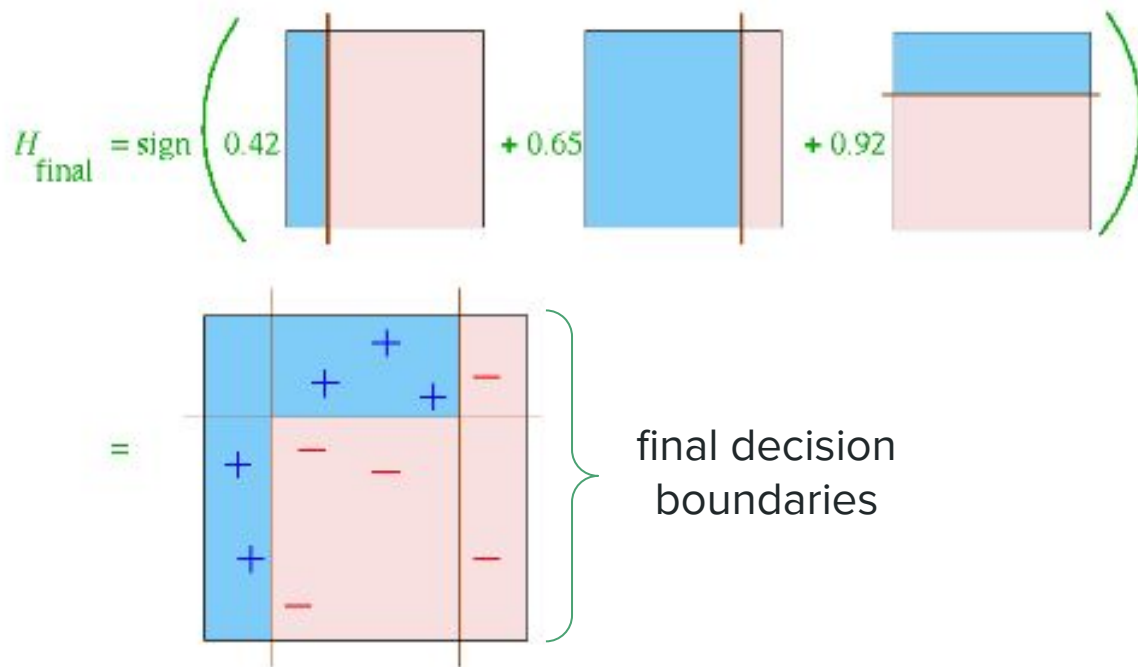
AdaBoost

A small example



AdaBoost

A small example



Introduction to modern optimization strategies

Objective & Loss Functions

Loss function: A derivable function that quantifies the error between true target y and prediction $\hat{y}$.

Usual loss functions: *Mean Square Error*

$$L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

Logarithmic Loss

$$L(y, \hat{y}) = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$$

Optimization Goal: Find predictions $\hat{y}$ that minimize total loss across dataset:

$$\arg \min_{\hat{y}} \sum_{i=1}^N L(y_i, \hat{y}_i)$$

Introduction to modern optimization strategies

Gradient Descent in parameters space

The Gradient ($\mathbf{g}$): Vector of partial derivatives indicating the best direction to increase the loss with respect to model parameters θ .

$$\mathbf{g} = \nabla_{\theta} \mathcal{L}(\theta) = \sum_{i=1}^N \frac{\partial L(y_i, f(x_i; \theta))}{\partial \theta}$$

Iterative parameters update: Step in the opposite direction of the gradient to minimize total loss:

$$\theta^{(m)} = \theta^{(m-1)} - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta^{(m-1)})$$

where $\eta > 0$ is a hyperparameter called *learning rate*.

Introduction to modern optimization strategies

Gradient Descent in function space (gradient boosting)

Shift in Perspective: Instead of updating fixed parameters θ , update the prediction function $F(x)$ directly:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Pseudo-Residuals (r_{im}): The new base estimator $h_m(x)$ is fit to the negative gradient:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$$

Gradient Boosting in a nutshell

Step 1: Refine with regularization

- Repeat steps 1-4 until the stopping condition is reached (usually the loss is no longer improving or the model is overfitting, preventing overfitting).

Step 2: Compute the residuals

- For each training example, compute the residual error by subtracting the predicted value from the actual value (always continuous values, even for classification!).
 - The goal is to correct the mistakes made by the previous model.
 - Identifies the samples for which the model needs improvement (similar to AdaBoost philosophy)

From AdaBoost to XGBoost: The evolution of Boosting

➤ Gradient Boosting Generalizing via gradients

- **Fixed Sample Weights** to function space potential of classifying observation y_i with w_i highly sensitive to noise and outliers.
- **Limited Tasks**, pseudo-residuals (negative classifications), making it hard to generalize to regression or multi-class tasks.

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$$

- **Universal framework**: Extends boosting to **any differentiable loss function** $L(y, F(x))$.

➤ XGBoost: Second-Order Optimization

- **2nd-Order Taylor Expansion**: Standard Gradient Boosting uses only 1st-order gradients (g_i). XGBoost incorporates 2nd-order Hessians (h_i) to capture loss surface curvature:

$$\mathcal{L}^{(m)} \approx \sum_{i=1}^N \left[g_i f_m(x_i) + \frac{1}{2} h_i f_m^2(x_i) \right] + \Omega(f_m)$$

$$\text{where } g_i = \frac{\partial L(y_i, \hat{y}^{(m-1)})}{\partial \hat{y}^{(m-1)}} \quad \text{and} \quad h_i = \frac{\partial^2 L(y_i, \hat{y}^{(m-1)})}{\partial (\hat{y}^{(m-1)})^2}$$

$\Omega(f_m) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$
regularization to penalize
the creation of a new leaf
this proposed in the
original XGBoost paper

- **Theoretical advantages**: Knowing both direction (g_i) and curvature (h_i) allows XGBoost to calculate exact optimal leaf values analytically, leading to significantly faster convergence and higher stability.

XGBoost

Chen and Guestrin, 2016.

Algorithm: XGBoost (Extreme Gradient Boosting)

Input: Dataset $\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N)\}$, Differentiable loss function $L(y, \hat{y})$

Number of trees M , Learning rate η , Regularization parameters λ, γ

Output: Ensemble model $F(x)$

Initialize predictions: $\hat{y}_i^{(0)} = \arg \min_c \sum_{i=1}^N L(y_i, c), \quad \forall i \in \{1, \dots, N\}$

For $m = 1$ to M **do** :

 Compute 1st-order gradients g_i and 2nd-order Hessians h_i for $i = 1, \dots, N$:

$$g_i = \left[\frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}_i} \right]_{\hat{y}_i = \hat{y}_i^{(m-1)}}, \quad h_i = \left[\frac{\partial^2 L(y_i, \hat{y}_i)}{\partial \hat{y}_i^2} \right]_{\hat{y}_i = \hat{y}_i^{(m-1)}}$$

 Construct tree structure $f_m(x)$ by finding splits that maximize gain:

$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma$$

 Compute optimal leaf weight w_j^* for each leaf node $j = 1, \dots, T$:

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

 Update model predictions for $i = 1, \dots, N$:

$$\hat{y}_i^{(m)} = \hat{y}_i^{(m-1)} + \eta \cdot f_m(x_i) \quad \left(\text{where } f_m(x_i) = w_{j(x_i)}^* \right)$$

End For

Return $F(x) = \hat{y}^{(0)}(x) + \eta \sum_{m=1}^M f_m(x)$

γ : Penalizes adding new leaf nodes to a tree (minimum gain threshold).

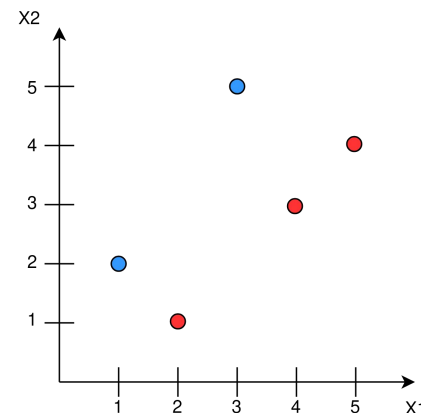
λ : Penalizes extreme prediction values in the leaves.

XGBoost: a small example

Sample i	X_1	X_2	Target y_i	Initial Prob $p_i^{(0)}$	Gradient $g_i = p_i^{(0)} - y_i$	Hessian $h_i = p_i^{(0)}(1 - p_i^{(0)})$
1	1.0	2.0	0	0.5	+0.5	0.25
2	2.0	1.0	1	0.5	-0.5	0.25
3	3.0	5.0	0	0.5	+0.5	0.25
4	4.0	3.0	1	0.5	-0.5	0.25
5	5.0	4.0	1	0.5	-0.5	0.25

Cross-entropy loss: $L(y_i, \hat{y}_i) = -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$

$$\text{where } p_i = \sigma(\hat{y}_i) = \frac{1}{1 + e^{-\hat{y}_i}}$$



XGBoost: a small example

Compute the similarity score:

$$\text{Score}_P = \frac{1}{2} \frac{G_P^2}{H_P + \lambda}$$

$$G_P = \sum_{i \in P} g_i \quad H_P = \sum_{i \in P} h_i$$

λ : L2 regularization parameter on leaf weights (usually fixed to 1)

$$G_P = \sum_{i=1}^5 g_i = 0.5 - 0.5 + 0.5 - 0.5 - 0.5 = -0.5$$

$$H_P = \sum_{i=1}^5 h_i = 5 \times 0.25 = 1.25$$

$$\text{Score}_P = \frac{1}{2} \frac{G_P^2}{H_P + \lambda} = \frac{1}{2} \frac{(-0.5)^2}{1.25 + 1.0} = \frac{0.25}{4.5} \approx \mathbf{0.0556}$$

Score is near 0, meaning there are opposing gradients that compensate each others

XGBoost: a small example

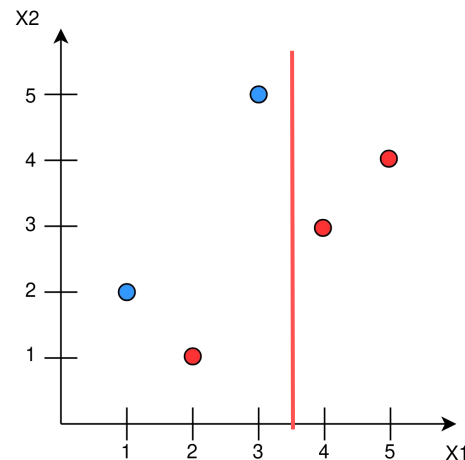
Evaluate competing splits

Candidate 1 : $X_1 < 3.5$

Left node: $G_L = 0.5, H_L = 0.75 \implies \text{Score}_L = \frac{1}{2} \frac{0.5^2}{0.75+1} \approx 0.0714$

Right node: $G_R = -1.0, H_R = 0.50 \implies \text{Score}_R = \frac{1}{2} \frac{(-1.0)^2}{0.50+1} \approx 0.3333$

$\text{Gain}(X_1 < 3.5) = 0.0714 + 0.3333 - 0.0556 = \mathbf{0.3491}$



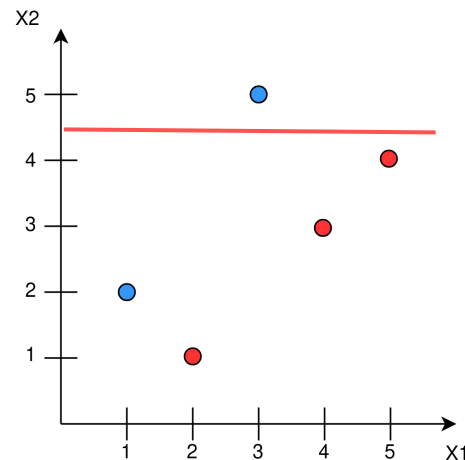
XGBoost: a small example

Evaluate competing splits

Candidate 2 : $X_2 < 4.5$

Left node: $G_L = -1.0, H_L = 1.0 \implies \text{Score}_L = \frac{1}{2} \frac{(-1.0)^2}{1.0+1} = 0.2500$

Right node: $G_R = 0.5, H_R = 0.25 \implies \text{Score}_R = \frac{1}{2} \frac{0.5^2}{0.25+1} = 0.1000$



$$\text{Gain}(X_2 < 4.5) = 0.2500 + 0.1000 - 0.0556 = \mathbf{0.2944}$$

$$\text{Gain}(X_1 < 3.5) = 0.0714 + 0.3333 - 0.0556 = \mathbf{0.3491}$$

XGBoost: a small example

Compute leaf weights

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

$$\text{Left leaf : } w_{\text{Left}}^* = -\frac{0.5}{0.75 + 1.0} = -\frac{0.5}{1.75} \approx -0.286$$

$$\text{Updated Prob } (\sigma(-0.286)) \approx \mathbf{0.429}$$

$$\text{Right leaf : } w_{\text{Right}}^* = -\frac{-1.0}{0.50 + 1.0} = \frac{1.0}{1.5} \approx +0.667$$

$$\text{Updated Prob } (\sigma(+0.667)) \approx \mathbf{0.661}$$

XGBoost: a small example

Updated tabular for Tree 2

Sample i	X_1	X_2	Target y_i	Updated Prob $p_i^{(1)}$	Gradient $g_i^{(1)} = p_i^{(1)} - y_i$	Hessian $h_i^{(1)} = p_i^{(1)}(1 - p_i^{(1)})$
1	1.0	2.0	0	0.429	+0.429	0.245
2	2.0	1.0	1	0.429	-0.571	0.245
3	3.0	5.0	0	0.429	+0.429	0.245
4	4.0	3.0	1	0.661	-0.339	0.224
5	5.0	4.0	1	0.661	-0.339	0.224

Repeat to build the second 2, and so on ...

Summary: AdaBoost vs XGBoost

	AdaBoost	XGBoost
Loss Function	Fixed exponential loss $L(y, \hat{y}) = e^{-y\hat{y}}$	Any 2nd-order differentiable loss function
Optimization	1st-order / Explicit sample weights w_i	2nd-order Taylor expansion (Gradients g_i & Hessians h_i)
Sample Focus	Increases explicit sample weights w_i on errors	High gradients g_i drive splits; Hessians h_i stabilize leaves
Leaf Outputs	Binary (± 1) scaled by global tree weight α_m	Continuous leaf values w_j^* computed locally per leaf
Regularization	Implicit (learning rate η , tree depth control)	Explicit structural penalty (γ) and leaf L_2 penalty (λ)
Outliers	High sensitivity (exponentially inflates errors)	Low to moderate (dampened by loss choice and λ)

Pushing further the scalability with LightGBM

Limitations of XGBoost

- **Exact Feature Pre-Sorting:** Sorting continuous values across all samples requires $O(N \log N)$ time per feature and massive memory to store pre-sorted indexes, causing severe speed and memory bottlenecks on large datasets.
- **Level-Wise Tree Growth:** Evaluates nodes layer-by-layer under a strict depth cap, spreading computational effort evenly across all active branches rather than aggressively deepening the specific high-error leaves that reduce loss fastest

Main idea of LightGBM: Scale gradient boosting to massive datasets by eliminating computational and memory bottlenecks without losing prediction accuracy.

LightGBM: Key innovations

- **Histogram-Based Binning:** Converts continuous features into 255 discrete integer bins. Reduces split search complexity from scanning every row $O(N)$ to just scanning bins $O(\text{bins})$.
- **GOSS (Gradient-based One-Side Sampling):** Retains 100% of high-gradient (large error) samples while randomly sampling low-gradient (small error) ones. Drastically cuts the row count evaluated per tree while keeping gain calculations unbiased.
- **EFB (Exclusive Feature Bundling):** Combines sparse, mutually exclusive features (like one-hot encoded columns) into single composite feature bins, drastically reducing effective feature count.
 - Basically, it optimizes the histogram construction by regrouping features that rarely take non-zero values simultaneously.
- **Leaf-Wise Tree Growth:** Expands trees by selecting and splitting only the single leaf across the entire tree that yields the highest loss reduction, driving down error much faster than rigid depth-capped growth.

EFB module on a small example

Feature *A* range is [0, 10]. Feature *B* range is [0, 20].

Bundle strategy: shift Feature *B*'s values by adding the the maximum value of Feature *A*.

Sample	Feature A (Original)	Feature B (Original)	Bundled Feature (Offset = 10)
1	5	0	5 (from A)
2	0	12	22 (12 + 10, from B)
3	8	0	8 (from A)
4	0	3	13 (3 + 10, from B)

LightGBM algorithm

Ke et al., 2017.

Algorithm: LightGBM (Gradient-Based One-Side Sampling & Leaf-Wise Tree Growth)

Inputs:

- Training dataset: $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$
- Loss function: $\mathcal{L}(y, \hat{y})$
- Number of boosting iterations: M
- Sampling ratios: Top gradient fraction $a \in (0, 1)$, low gradient fraction $b \in (0, 1)$
- Learning rate: η , Regularization parameter: λ

Procedure:

same as XGBoost

1. Initialize Base Model:

$$f_0(x) = \arg \min_c \sum_{i=1}^N \mathcal{L}(y_i, c)$$

2. For $m = 1$ to M do:

a. Compute First & Second Derivatives:

$$g_i = \left. \frac{\partial \mathcal{L}(y_i, f(x_i))}{\partial f(x_i)} \right|_{f(x_i)=f_{m-1}(x_i)}, \quad h_i = \left. \frac{\partial^2 \mathcal{L}(y_i, f(x_i))}{\partial f(x_i)^2} \right|_{f(x_i)=f_{m-1}(x_i)}$$

b. GOSS Data Sampling:

- Sort dataset instances by absolute gradient values $|g_i|$ in descending order.
- Select top $a \cdot N$ instances with largest gradients:
 $A =$ instances with top $a \cdot N$ gradients
- From remaining set A^c of size $(1-a) \cdot N$, randomly sample $b \cdot N$ instances:
 $B \subset A^c, |B| = b \cdot N$
- Form sampled subset $I = A \cup B$.

- Assign sample weight correction factor:

$$w_i = \begin{cases} 1 & \text{if } i \in A \\ \frac{1-a}{b} & \text{if } i \in B \end{cases}$$

c. Histogram Construction & Leaf-Wise Splitting:

- Initialize tree root with sampled subset I .
- For each candidate split d on binned feature j , compute modified variance gain:

$$\tilde{V}_j(d) = \frac{1}{|I|} \left[\frac{(\sum_{i \in I_L} w_i g_i)^2}{\sum_{i \in I_L} w_i h_i + \lambda} + \frac{(\sum_{i \in I_R} w_i g_i)^2}{\sum_{i \in I_R} w_i h_i + \lambda} - \frac{(\sum_{i \in I} w_i g_i)^2}{\sum_{i \in I} w_i h_i + \lambda} \right]$$

- Choose leaf node with the **maximum delta loss reduction** (Leaf-Wise Growth strategy) and split it.

d. Compute Leaf Outputs:

- For each leaf k in constructed tree T_m :

$$w_k^* = - \frac{\sum_{i \in \text{Leaf}_k} w_i g_i}{\sum_{i \in \text{Leaf}_k} w_i h_i + \lambda}$$

e. Update Model:

$$f_m(x) = f_{m-1}(x) + \eta \sum_k w_k^* \mathbb{I}(x \in \text{Leaf}_k)$$

3. Return Ensembled Model: $f_M(x)$

a : top rate hyperparameter.

Fraction of data to keep containing the largest absolute gradients (default: 0.2)

select the worst classified samples

select among the other samples

b : other rate hyperparameter.

Fraction of the remaining small gradient data to randomly sample. (default: 0.1)

Assign a weight to samples, spending if they are among the worst classified or not

Discretize the continuous feature spaces into histograms to search the best split

update the leafs weights, same as XGBoost

Depth-wise vs Leaf-wise growth

Feature	Depth-Wise (Level-Wise)	Leaf-Wise (Best-First)
Primary Framework	XGBoost (traditional default)	LightGBM
Expansion Rule	Splits all leaves at depth d before depth $d + 1$	Splits only the single leaf with maximum Gain
Tree Structure	Balanced, symmetric trees	Asymmetric, deep trees
Key Hyperparameter	<code>max_depth</code>	<code>num_leaves</code>
Computational Focus	Computes splits across all nodes at current level	Allocates compute exclusively to high-residual nodes
Overfitting Risk	Lower on smaller or noisy datasets	Higher on small datasets if <code>num_leaves</code> is large

Summary: XGBoost vs LightGBM

Tree growth strategy

- **XGBoost (Level-wise/Depth-wise):** Splits nodes level by level across the entire tree depth at each step. This creates balanced trees and controls model complexity, but can waste computation on splits with low loss reduction.
- **LightGBM (Leaf-wise):** Splits the single leaf node that yields the maximum reduction in loss, regardless of the tree depth. This creates deeper, asymmetrical trees that optimize the objective function faster.

Efficiency & speed

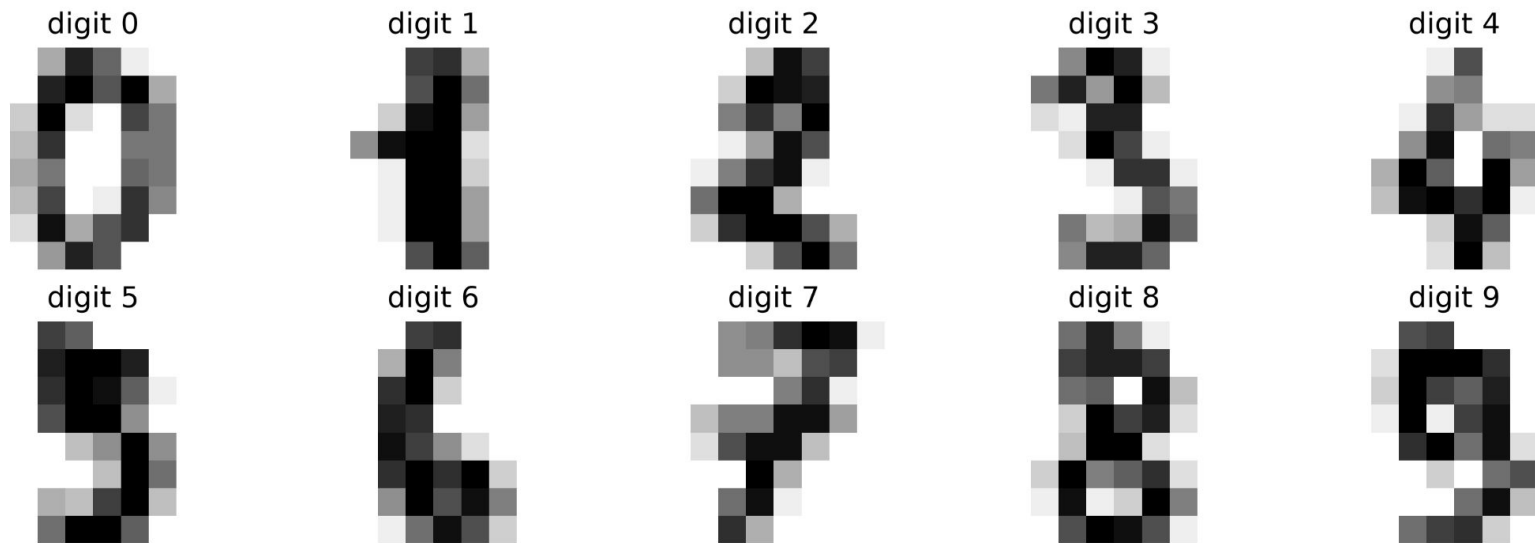
- **XGBoost:** Pre-sorts feature values and uses exact split finding algorithms, leading to higher memory usage and slower training times when the number of data increases.
- **LightGBM:** Uses histogram-based algorithms and Gradient-based One-Side Sampling (GOSS) to bin continuous features and downsample gradients, offering faster execution and reduced memory usage.

Overfitting & scaling

- **XGBoost:** Less prone to overfitting on smaller or noisier datasets due to its controlled level-wise balanced structure.
- **LightGBM:** Highly sensitive to overfitting on smaller datasets because leaf-wise growth can lead to extreme depth (such as CART without max depth limit). It relies heavily on hyperparameters like the *maximum depth* and the *minimum data per leaf*.

Back to the small example: Digit dataset

Handwritten digits - 8x8 grayscale (pixel values 0-16)



1797 samples (1258 for training and 539 for testing).

64 features (1 per pixel).

Example: Digit dataset

	CART	Bagging	Random Forest	Extra Trees	AdaBoost	XGBoost	LightGBM
train acc.	100%	100%	100%	100%	100%	100%	100%
test acc.	85.2%	92,2%	96.7%	97.4%	92.96%	95%	95.93%
OOB error	-	4.38%	2.70%	-	-	-	-

max depth: 4

The dataset is quite small and well separated, so the variance reduction induced by the bagging strategy (Random Forest and Extra Trees) is well suited for this dataset.

Summary and conclusion

Comparison matrix

Dimension	Bagging (Bootstrap Aggregating)	Boosting
Core Concept	Builds models in parallel on independent bootstrap samples of the dataset.	Builds models sequentially , where each tree learns from the errors of previous trees.
Primary Goal	Variance Reduction (controls overfitting without changing bias).	Bias Reduction (combines multiple weak learners into a strong model).
Base Learner Type	Complex & Deep (e.g., fully grown, high-variance decision trees).	Simple & Shallow (e.g., decision stumps or shallow trees with depth 3–6).
Data Resampling	Uniform random sampling with replacement (Bootstrap sampling).	Reweights misclassified points (AdaBoost) or targets residual gradients g_i (Gradient Boosting).
Aggregation Method	Simple majority voting (classification) or unweighted mean (regression).	Weighted sum of predictions: $\hat{y}_i = \sum_{m=1}^M \eta f_m(x_i)$.
Parallelization	High: Base models train completely independently across CPU/GPU cores.	Low (Sequential Structure): Optimized via feature-level parallelization (XGBoost/LightGBM).
Advantages	- Extremely robust against noise and outliers. - Unlikely to overfit as more trees are added. - Works out of the box with minimal hyperparameter tuning.	- High predictive accuracy on complex tabular datasets. - Flexible optimization via custom loss functions. - Fast inference speed due to shallow trees.
Drawbacks	- Cannot resolve high structural bias (underfitting). - Predictions can be redundant if trees are highly correlated.	- Vulnerable to noisy data and mislabeled outliers. - Requires careful hyperparameter tuning (<code>learning_rate</code> , <code>max_depth</code> , <code>num_leaves</code>).
Key Algorithms	Random Forest, Extra Trees, Bagged Decision Trees.	AdaBoost, Gradient Boosting (GBDT), XGBoost, LightGBM.

Practical advice

Scenario / Constraint	Recommended Model	Primary Reason
Small Datasets or Standard Baseline	Random Forest	Highly resistant to overfitting; reliable out-of-the-box baseline requiring minimal hyperparameter tuning.
High Feature Noise or Need for Faster Bagging	Extra Trees	Randomizes split thresholds, reducing variance and training time beyond standard Random Forest.
Simple Binary Tasks & Teaching Boosting Basics	AdaBoost	Uses simple decision stumps to intuitively demonstrate sequential re-weighting of misclassified samples.
Medium Tabular Data & Maximum Metric Precision	XGBoost	Fine-grained regularization controls (L1/L2 penalties, depth pruning) yield peak predictive accuracy.
Large-Scale Data & Speed-Critical Iteration	LightGBM	GOSS sampling and histogram binning deliver $3 \times$ -- $5 \times$ speedups over standard GBDT models.

Practical guidance

- **Establish a Bagging Baseline First:** Always begin with Random Forest or Extra Trees. They handle complex interactions without overfitting and provide a strong benchmark. Choose Extra Trees if you suspect heavy continuous feature noise or need faster fit times.
- **Scale with Gradient Boosting:** Use XGBoost when running extensive hyperparameter optimization loops on medium datasets. Shift to LightGBM as soon as training memory or runtime becomes a bottleneck.
- **Tree Count vs. Overfitting:** Adding trees to Random Forest or Extra Trees reduces prediction variance without overfitting. In AdaBoost, XGBoost, and LightGBM, adding excessive trees *will* overfit unless you scale down the learning rate.
- **LightGBM Leaf-Wise Caution:** Always constrain `num_leaves` to be strictly less than $2^{\text{max_depth}}$ on smaller datasets to prevent leaf-wise growth from creating asymmetrical, overfitted branches.